Zustand : partager un état client

I. Partager un état entre composants éloignés

Le problème du prop drilling

Le portfolio propose un mode jour/nuit. Le bouton vit dans l'en-tête. Dans une SPA React, l'état du thème est détenu par `App` et doit traverser chaque niveau jusqu'à lui.

```
App              détient l'état : theme = "dark"
└── Layout       reçoit theme, ne l'utilise pas
    └── Header   reçoit theme, ne l'utilise pas
        └── ThemeToggle   utilise theme
```

```
const App = () => {
  const [theme, setTheme] = useState<"light" | "dark">("light");
  return <Layout theme={theme} onToggle={() => setTheme(theme === "light" ? "dark" : "light")} />;
};
```

Les composants intermédiaires transportent une donnée qui ne les concerne pas. Toute modification de signature impacte la chaîne entière.

Sous Next.js, c'est pire. Le parent commun est `RootLayout`, un Server Component. Il n'a pas d'état, et une fonction `onToggle` ne traverse pas la frontière serveur-client.

React - Passer des données en profondeur avec le contexte

Où placer chaque état

Un store ne remplace pas les autres outils. Chaque état a sa place.

APPROCHE	EXEMPLES	QUAND L'UTILISER
`useState`	« Lien copié », menu ouvert	État d'un seul composant
Context natif	Langue, utilisateur connecté	Donnée rarement modifiée
Store Zustand	Thème, préférences d'affichage	État client lu par des composants éloignés
Server Component	Articles, projets	Données Supabase
URL, `searchParams`	Recherche, page courante	État partageable par un lien

Un contenu Supabase copié dans un store crée une seconde source de vérité, qui finit par diverger du serveur. Un filtre rangé dans un store perd le partage de lien et le bouton Retour.

React - Partager l'état entre composants

Pourquoi Zustand

Un Context React résout le prop drilling. Chaque changement de valeur rend de nouveau tous ses consommateurs, ce qui convient à une donnée rarement modifiée.

Zustand range l'état hors de React. Chaque composant s'abonne à la seule portion qu'il affiche.

CRITÈRE	REDUX TOOLKIT	ZUSTAND	SIGNALS (ANGULAR)
Code à écrire	Slice, actions, selectors	Une fonction `create`	`signal`, `computed`
Provider requis	Oui	Non	Injection de dépendances
Re-renders	Par sélecteur	Par sélecteur	Par signal lu
Persistance	Bibliothèque externe	Middleware `persist` intégré	À écrire

Le store vit uniquement dans des Client Components : un Server Component ne lit jamais un store.

```
npm install zustand
```

Zustand - Introduction · Zustand - Comparison

II. Les fondamentaux de Zustand

Premier store avec `create`

`create` reçoit une fonction qui renvoie l'état initial et ses actions. Il retourne un hook React.

```ts
// src/stores/prefs-store.ts
import { create } from "zustand";

export type Theme = "light" | "dark";

export type PrefsStore = {
  theme: Theme;
  toggleTheme: () => void;
};

export const usePrefsStore = create<PrefsStore>()((set) => ({
  theme: "light",
  toggleTheme: () => set((state) => ({ theme: state.theme === "light" ? "dark" : "light" })),
}));
```

`set` reçoit l'état précédent et renvoie les propriétés modifiées. Zustand les fusionne avec le reste de l'état, au premier niveau. Les actions vivent dans le store : un composant n'écrit jamais l'état lui-même.

Zustand - Updating state · Zustand - Guide TypeScript

Lire et modifier le store hors de React

Le hook expose aussi `getState`, `setState` et `subscribe`. Le store se teste donc dans un simple script Node.js.

```ts
// demo.ts
import { create } from "zustand";

type PrefsStore = {
  theme: "light" | "dark";
  fontSize: number;
  toggleTheme: () => void;
};

const usePrefsStore = create<PrefsStore>()((set, get) => ({
  theme: "light",
  fontSize: 16,
  toggleTheme: () => set((state) => ({ theme: state.theme === "light" ? "dark" : "light" })),
}));

usePrefsStore.subscribe((state, previous) => console.log(previous.theme, "->", state.theme));
usePrefsStore.getState().toggleTheme();
usePrefsStore.setState({ fontSize: 20 });
console.log(usePrefsStore.getState());
```

`get` lit l'état courant dans une action, au moment de l'appel.

> ***TP 5 min*** *: ajoutez au store une action* `reset` *qui affiche le thème courant avec* `get`*, puis remet* `theme` *à* `"light"` *et* `fontSize` *à* `16`*. Appelez-la à la fin du script et vérifiez la sortie.*

Zustand - create

Consommer le store dans un Client Component

Un composant appelle le hook avec un **sélecteur**, une fonction qui extrait la portion utile. Aucun Provider n'est nécessaire.

```tsx
// src/components/ThemeToggle.tsx
"use client";

import { usePrefsStore, type PrefsStore } from "@/stores/prefs-store";

export function ThemeToggle() {
  const theme = usePrefsStore((state: PrefsStore) => state.theme);
  const toggleTheme = usePrefsStore((state: PrefsStore) => state.toggleTheme);

  return (
    <button type="button" aria-pressed={theme === "dark"} onClick={toggleTheme}>
      Mode sombre
    </button>
  );
}
```

Le libellé reste fixe : `aria-pressed` annonce l'état du bouton bascule au lecteur d'écran. Le layout reste un Server Component et place `<ThemeToggle />` dans sa navigation. Seul le bouton part dans le bundle.

> 🛠️ ***TP (~10 min)*** *: créez le store et le bouton, puis ajoutez-le dans la navigation de* `src/app/layout.tsx`*. Inspectez le bouton dans l'onglet Éléments :* `aria-pressed` *change-t-il au clic ? La page change-t-elle de couleur ?*

Zustand - Introduction · MDN - aria-pressed

Sélecteurs et re-renders

Après chaque modification, Zustand exécute le sélecteur sur l'ancien et le nouvel état, puis compare les deux résultats avec `Object.is`.

```
Mutation du store
     ↓
sélecteur(ancien état)  → previousValue
sélecteur(nouvel état)  → nextValue
     ↓
Object.is(previousValue, nextValue) ?
  oui → aucun re-render
  non → re-render du composant abonné
```

```ts
// Subscribes to the whole store: re-renders on every change, even fontSize
const prefs = usePrefsStore();

// Re-renders only when theme changes
const theme = usePrefsStore((state: PrefsStore) => state.theme);

// Actions keep the same reference: always stable
const toggleTheme = usePrefsStore((state: PrefsStore) => state.toggleTheme);
```

Une chaîne, un nombre ou une action se comparent sans surprise. Un sélecteur qui construit un objet ou un tableau renvoie une nouvelle référence à chaque appel.

Zustand - Prevent rerenders with useShallow

`useShallow` pour plusieurs valeurs

`useShallow` compare les propriétés de l'objet renvoyé une à une, au lieu de sa référence.

```
import { useShallow } from "zustand/react/shallow";

// New object on every call: render loop
const unstable = usePrefsStore((state) => ({
  theme: state.theme,
  toggleTheme: state.toggleTheme,
}));

// Same values, same result: stable
const { theme, toggleTheme } = usePrefsStore(
  useShallow((state) => ({
    theme: state.theme,
    toggleTheme: state.toggleTheme,
  })),
);
```

Sans `useShallow`, la page plante dès le premier rendu :

```
The result of getServerSnapshot should be cached to avoid an infinite loop
Uncaught Error: Maximum update depth exceeded.
```

> 🛠️ ***TP (~5 min)*** *: dans `ThemeToggle`, remplacez les deux sélecteurs par un seul qui renvoie un objet, sans `useShallow`. Lisez la console, puis corrigez avec `useShallow`.*

Zustand - useShallow

Appliquer le thème avec `subscribe`

Le thème doit être une propriété sur `<html>`, que le CSS lit. `subscribe` réagit à chaque changement sans provoquer de re-render React.

```tsx
// src/components/ThemeSync.tsx
"use client";

import { useEffect } from "react";
import { usePrefsStore, type PrefsStore } from "@/stores/prefs-store";

export function ThemeSync() {
  useEffect(() => {
    document.documentElement.dataset.theme = usePrefsStore.getState().theme;
    return usePrefsStore.subscribe((state: PrefsStore) => {
      document.documentElement.dataset.theme = state.theme;
    });
  }, []);

  return null;
}
```

```css
/* src/app/globals.css: replaces the existing :root rule */
:root {
  --color-text: #182028;
  --color-bg: #f5f2ea;
  color-scheme: light;
  color: var(--color-text);
  background: var(--color-bg);
  font-family: Inter, system-ui, sans-serif;
}

:root[data-theme="dark"] {
  --color-text: #ece8df;
  --color-bg: #14181d;
  color-scheme: dark;
}
```

`ThemeSync` ne rend rien. Le layout le place une fois dans `<body>`. L'effet applique d'abord la valeur courante, puis chaque changement. La fonction renvoyée par `subscribe` désabonne le composant au démontage.

> 🛠️ ***TP (~10 min)*** *: ajoutez `ThemeSync` au layout et remplacez chaque couleur de `globals.css` par une variable. Le bouton bascule-t-il la page ? Rechargez : que devient le thème ?*

Zustand - create · MDN - color-scheme

III. Persister le thème avec Next.js

`persist` enregistre dans `localStorage`

`persist` enveloppe la fonction du store. Il enregistre l'état à chaque modification et le relit au chargement. `partialize` choisit ce qui est enregistré.

```ts
// src/stores/prefs-store.ts
import { create } from "zustand";
import { createJSONStorage, persist } from "zustand/middleware";

export type Theme = "light" | "dark";

export type PrefsStore = {
  theme: Theme;
  toggleTheme: () => void;
};

export const usePrefsStore = create<PrefsStore>()(
  persist(
    (set) => ({
      theme: "light",
      toggleTheme: () => set((state) => ({ theme: state.theme === "light" ? "dark" : "light" })),
    }),
    {
      name: "theme",
      storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({ theme: state.theme }),
    },
  ),
);
```

La clé `theme` contient `{"state":{"theme":"dark"},"version":0}`. Les actions ne sont pas enregistrées : `JSON.stringify` ignore les fonctions.

Zustand - persist

Le serveur ne connaît pas `localStorage`

Sur le serveur, `localStorage` n'existe pas. `persist` se désactive et la page est rendue en mode clair, avec `aria-pressed="false"`.

Dans le navigateur, `persist` relit `localStorage` dès l'import du store. Le premier rendu doit pourtant produire le même HTML que le serveur, sinon React signale une erreur d'hydratation. Zustand le garantit : pendant l'hydratation, `usePrefsStore` renvoie l'état initial du store, puis rend de nouveau le composant avec la valeur relue.

```
Serveur          theme: "light"   HTML en mode clair
Hydratation      theme: "light"   même rendu, aucun écart
Après            theme: "dark"    ThemeToggle se rend de nouveau, ThemeSync applique data-theme
```

Le thème est correct, mais seulement une fois le JavaScript chargé et exécuté. D'ici là, le visiteur voit la page en clair.

> 🛠️ ***TP (~10 min)*** *: ajoutez `persist`, passez en mode sombre, puis rechargez. Le thème survit-il ? Rechargez avec le réseau ralenti (onglet Réseau, « Slow 4G ») et observez le premier affichage. Lisez la clé `theme` dans l'onglet Application.*

Zustand - Persisting store data · React - hydrateRoot

Supprimer le flash avec un script inline

Le JavaScript de la page s'exécute après le premier affichage. Pour appliquer le thème avant, un court script placé dans `<head>` lit `localStorage` pendant la lecture du HTML.

```
// src/app/layout.tsx (extrait)
const themeScript = `try {
  const theme = JSON.parse(localStorage.getItem("theme")).state.theme;
  if (theme === "light" || theme === "dark") document.documentElement.dataset.theme = theme;
} catch {}`;

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="fr" data-theme="light" suppressHydrationWarning>
      <head>
        <script dangerouslySetInnerHTML={{ __html: themeScript }} />
      </head>
      <body>
        {/* header with ThemeToggle, then children */}
        {children}
        <ThemeSync />
      </body>
    </html>
  );
}
```

Le script modifie `data-theme` avant l'hydratation. `suppressHydrationWarning` indique à React que cet écart sur `<html>` est voulu. Il ne s'applique qu'à cet élément, pas à ses enfants.

Le layout reste statique : aucune route ne change de symbole dans `npm run build`. Lire un cookie avec `cookies()` donnerait le thème au serveur, mais rendrait chaque page dynamique.

> 🛠️ ***TP (~10 min)*** *: ajoutez le script, puis rechargez en mode sombre avec « Slow 4G ».*

Next.js - Preventing flash before hydration